Value Added Problem : 1

Problem Statement 1 :-

Given a Binary Search Tree and a range [low, high]. Find all the numbers in the BST that lie in the given range. Note: Element greater than or equal to root go to the right side.
Concept: Binary Search Tree

Code:

```
#include <stdio.h>
#include <stdlib.h>

void displayInfo() {
   printf("Name : Kashti Gupta\n");
   printf("PRN : 25070521200\n");
   printf("Section : D-1\n");
   printf("Value Added Problem : 1\n");
   printf("---------------------------\n");
}

struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) exit(1);
   newNode->data = data;
   newNode->left = newNode->right = NULL;
   return newNode;
}

struct Node* insert(struct Node* root, int data) {
   if (root == NULL) return createNode(data);

   if (data < root->data)
      root->left = insert(root->left, data);
   else
      root->right = insert(root->right, data);

   return root;
}

void printInRange(struct Node* root, int low, int high) {
   if (root == NULL) return;

   if (root->data > low)
      printInRange(root->left, low, high);

   if (root->data >= low && root->data <= high)
      printf("%d ", root->data);

   if (root->data < high)
      printInRange(root->right, low, high);
}

int main() {
```

```
    displayInfo();

    struct Node* root = NULL;

    root = insert(root, 17);
    insert(root, 4);
    insert(root, 18);
    insert(root, 2);
    insert(root, 9);

    int low = 4, high = 24;
    printf("Output: ");
    printInRange(root, low, high);
    printf("\n");

    return 0;
}
```

Execution:

Programiz
C Online Compiler

Programiz PRO ›

main.c | Share | Run

```c
#include <stdio.h>
#include <stdlib.h>

void displayInfo() {
    printf("Name : Kashti Gupta\n");
    printf("PRN : 25070521200\n");
    printf("Section : D-1\n");
    printf("Value Added Problem : 1\n");
    printf("--------------------------\n");
}

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
        Node));
    if (newNode == NULL) exit(1);
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

Output | Clear

```
Name : Kashti Gupta
PRN : 25070521200
Section : D-1
Value Added Problem : 1
--------------------------
Output: 4 9 17 18


=== Code Execution Successful ===
```

Value Added Problem : 2

Problem Statement 2 :-

Given a string s, composed of different combinations of '(' , ')', '{', '}', '[', ']'. Determine whether the Expression is balanced or not.
An expression is balanced if:
1. Each opening bracket has a corresponding closing bracket of the same type.
2. Opening brackets must be closed in the correct order.
Concept: Stack

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 1000000

char stack[MAX];
int top = -1;

void push(char c) {
   stack[++top] = c;
}

char pop() {
   if (top == -1)
      return '\0';
   return stack[top--];
}

int isMatchingPair(char open, char close) {
   if (open == '(' && close == ')') return 1;
   if (open == '{' && close == '}') return 1;
   if (open == '[' && close == ']') return 1;
   return 0;
}

int isBalanced(char *s) {
   for (int i = 0; s[i] != '\0'; i++) {
      if (s[i] == '(' || s[i] == '{' || s[i] == '[') {
         push(s[i]);
      }
      else if (s[i] == ')' || s[i] == '}' || s[i] == ']') {
         if (top == -1)
            return 0;

         char topElement = pop();
```

```
            if (!isMatchingPair(topElement, s[i]))
                return 0;
        }
    }
    return (top == -1);
}

int main() {
    printf("Name : Kashti Gupta\n");
    printf("PRN : 25070521200\n");
    printf("Section : D-1\n");
    printf("Value Added Problem : 2\n");
    printf("--------------------------\n");

    char s[MAX];
    printf("Enter expression: ");
    scanf("%s", s);

    if (isBalanced(s))
        printf("true\n");
    else
        printf("false\n");

    return 0;
}
```

Execution:

Programiz
C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#include <string.h>

#define MAX 1000000

char stack[MAX];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
    if (top == -1)
        return '\0';
    return stack[top--];
}

int isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
    return 0;
}

```

Output

Clear

```
Name : Kashti Gupta
PRN : 25070521200
Section : D-1
Value Added Problem : 2
--------------------------
Enter expression: ()
true


=== Code Execution Successful ===
```